

MI VDF API

Version 2.09

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.03	Initial release	11/12/2018
2.04	Document refinement	04/12//2019
2.05	- Add Work Flow - Fix MD param info	10/12/2020
2.06	- Fix VG param info. - Add description of new structure.	11/02/2020
2.07	- Add Flowchart Title - Add MI_MD_ResultSize_t struct - Fix variables and funcs hyperlinks - Add MI_OD_Result_t struct member - Add proc debug info	11/10/2021
2.08	Change 16RstBufSize to u32RstBufSize	03/28/2022
2.09	Add 64bit physical address of MD/OD	04/12/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. OVERVIEW.....	1
1.1. Module Description.....	1
1.2. Flow Chart.....	1
1.3. Keyword Description.....	3
2. API REFERENCE.....	4
2.1. VDF API List.....	4
2.2. VDF API Description.....	4
2.2.1 MI_VDF_Init.....	4
2.2.2 MI_VDF_Uninit.....	5
2.2.3 MI_VDF_CreateChn.....	6
2.2.4 MI_VDF_DestroyChn.....	6
2.2.5 MI_VDF_SetChnAttr.....	7
2.2.6 MI_VDF_GetChnAttr.....	8
2.2.7 MI_VDF_EnableSubWindow.....	9
2.2.8 MI_VDF_Run.....	10
2.2.9 MI_VDF_Stop.....	10
2.2.10 MI_VDF_GetResult.....	11
2.2.11 MI_VDF_PutResult.....	12
2.2.12 MI_VDF_GetLibVersion.....	13
2.2.13 MI_VDF_GetDebugInfo.....	14
3. DATA TYPE.....	15
3.1. VDF Structure List.....	15
3.2. VDF Structure Description.....	15
3.2.1 MI_VDF_WorkMode_e.....	15

3.2.2	MI_VDF_Color_e.....	16
3.2.3	MI_VDF_ODWindow_e.....	16
3.2.4	MI_MD_Result_t.....	17
3.2.5	MI_OD_Result_t.....	18
3.2.6	MI_VG_Result_t.....	19
3.2.7	MI_VDF_Result_t.....	20
3.2.8	MI_VDF_MdAttr_t.....	20
3.2.9	MI_VDF_OdAttr_t.....	21
3.2.10	MI_VDF_VgAttr_t.....	22
3.2.11	MI_VDF_ChnAttr_t.....	23
3.2.12	MDRST_STATUS_t.....	24
3.2.13	MI_MD_ResultSize_t.....	25
3.3.	MD Structure List.....	25
3.4.	MD Structure Description.....	26
3.4.1	MDMB_MODE_e.....	26
3.4.2	MDSAD_OUT_CTRL_e.....	26
3.4.3	MDALG_MODE_e.....	27
3.4.4	MDCCL_ctrl_t.....	27
3.4.5	MDPreproc_ctrl_t.....	28
3.4.6	MDblock_info_t.....	28
3.4.7	MDPoint_t.....	29
3.4.8	MDROI_t.....	29
3.4.9	MDSAD_DATA_t.....	30
3.4.10	MDOBJ_t.....	30
3.4.11	MDOBJ_DATA_t.....	31
3.4.12	MI_MD_IMG_t.....	32
3.4.13	MI_MD_IMG64_t.....	32

3.4.14	MI_MD_static_param_t.....	33
3.4.15	MI_MD_param_t.....	33
3.5.	OD Structure List.....	34
3.6.	OD Structure Description.....	35
3.6.1	MI_OD_WIN_STATE.....	35
3.6.2	ODColor_e.....	35
3.6.3	ODWindow_e.....	36
3.6.4	ODPoint_t.....	36
3.6.5	ODROI_t.....	37
3.6.6	MI_OD_IMG_t.....	37
3.6.7	MI_OD_IMG64_t.....	38
3.6.8	MI_OD_static_param_t.....	38
3.6.9	MI_OD_param_t.....	39
3.7.	VG Structure List.....	40
3.8.	VG Structure Description.....	40
3.8.1	VgFunction.....	40
3.8.2	VgRegion_Dir.....	41
3.8.3	VgSize_Sensitively.....	41
3.8.4	VgDirection_State.....	42
3.8.5	MI_VG_Point_t.....	43
3.8.6	MI_VgLine_t.....	43
3.8.7	MI_VgRegion_t.....	43
3.8.8	MI_VgSet_t.....	44
3.8.9	MI_VgResult_t.....	46
3.8.10	MI_VgBoundingBox_t.....	46
3.8.11	MI_VgDetectThd.....	47
4.	ERROR CODE.....	48

5. PROCFS INTRODUCTION.....50

5.1. cat..... 50

5.2. echo..... 59

s

1. OVERVIEW

1.1. Module Description

VDF realizes the functions of MD/OD/ VG video channel initialization, channel management, video detection result management and channel destruction.

1.2. Flow Chart

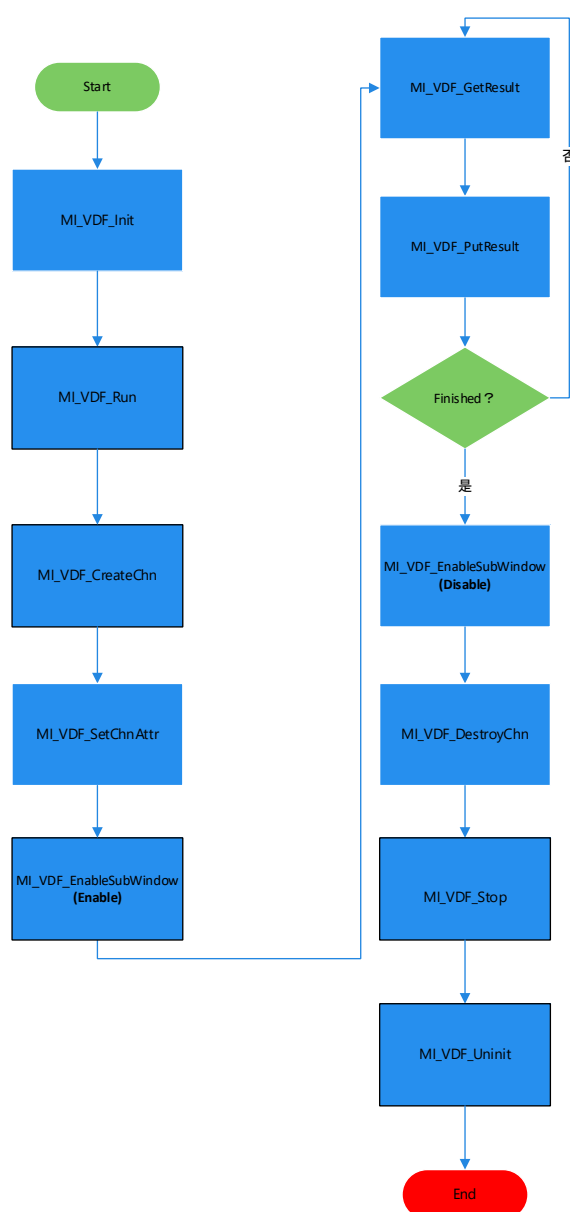


Figure 1-1 Function call flow of a VDF channel

The above figure is the call process of opening a VDF, but VDF can support multiple channels, and each channel can be set to one of MD/OD/VG working modes.

Each VDF channel must be controlled by itself to call MI_VDF_CreateChn, MI_VDF_SetChnAttr, MI_VDF_EnableSubWindow, MI_VDF_GetResult, MI_VDF_PutResult, MI_VDF_DestroyChn independently.

As shown below, four VDF channels are enabled, among which channel 0 and channel 1 are set to MD mode and channel 2 and channel 3 are set to OD mode.

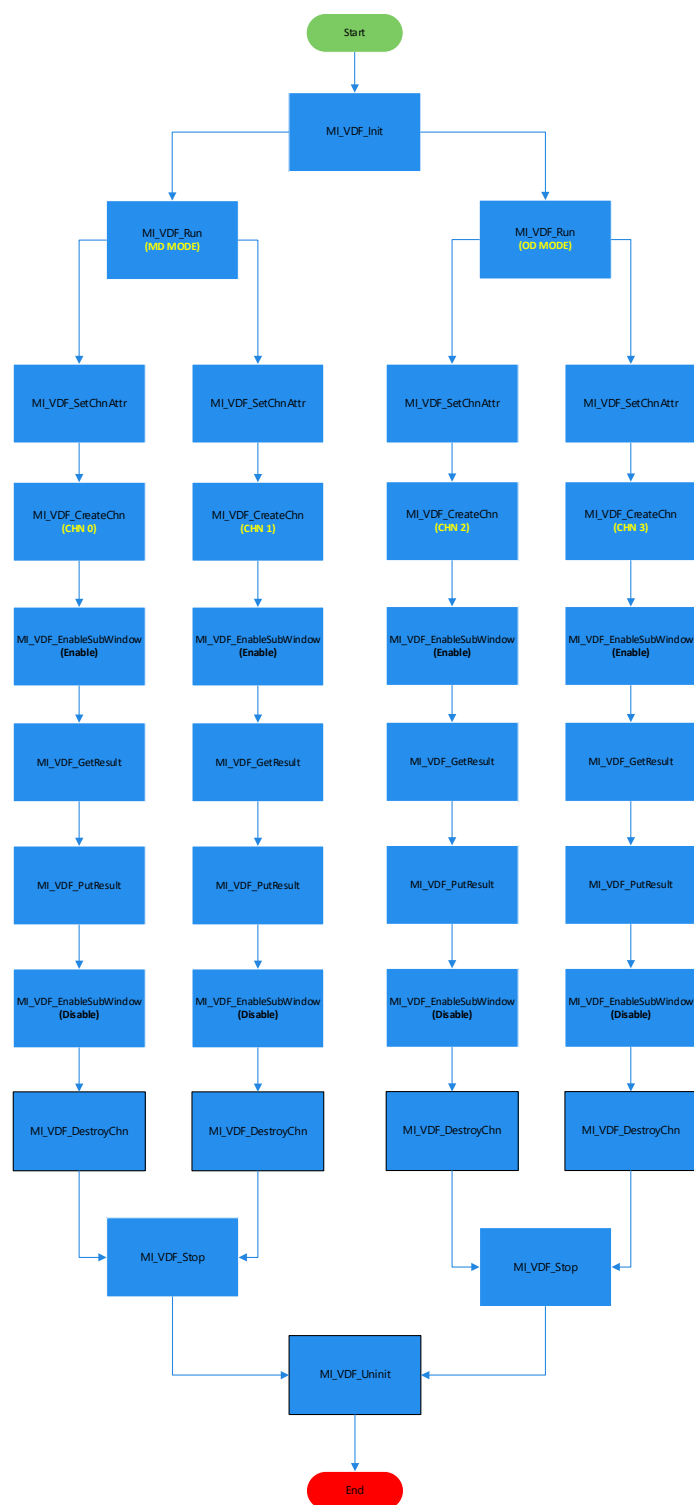


Figure 1-2 Function call flow of 4 VDF channels

1.3. Keyword Description

- MD: Movement detection

- OD: Occlusion detection
- VG: Virtual fence

2. API REFERENCE

2.1. VDF API List

API Name	Function
MI_VDF_Init	Initialize MI_VDF module
MI_VDF_Uninit	Un-initialize MI_VDF Module
MI_VDF_CreateChn	Create channel for MD/OD/VG
MI_VDF_DestroyChn	Destroy created channel MD/OD/VG
MI_VDF_SetChnAttr	Set channel (MD/OD/VG) attribute
MI_VDF_GetChnAttr	Get attribute from channel (MD/OD/VG)
MI_VDF_EnableSubWindow	Enable sub windows for (MD/OD/VG) channel
MI_VDF_Run	Start detection for channel (MD/OD/VG)
MI_VDF_Stop	Stop detection for channel (MD/OD/VG)
MI_VDF_GetResult	Get detection result from channel (MD/OD/VG)
MI_VDF_PutResult	Release detection result from channel (MD/OD/VG)
MI_VDF_GetLibVersion	Get version number of the specified channel (MD/OD/VG)
MI_VDF_GetDebugInfo	Get debug info of the specified channel. (Only VG support this feature)

2.2. VDF API Description

2.2.1 MI_VDF_Init

➤ Functions

Initialize the VDF module.

➤ Syntax

```
MI_S32 MI_VDF_Init(void);
```

➤ Parameters

N/A.

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- MI_VDF_Init needs to be called after calling MI_SYS_Init.
- It is not possible to call MI_VDF_Init repeatedly.

➤ Example

N/A

➤ Related APIs

[MI_VDF_Uninit](#)

2.2.2 MI_VDF_Uninit

➤ Functions

Destructing the MI_VDF system, before calling MI_VDF_Uninit, you need to ensure that the created VDF channels have been disabled and the video detection results of all channels are

released

➤ Syntax

MI_S32 MI_VDF_Uninit (void);

➤ Parameters

N/A

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- Before the MI_VDF_Uninit call, you need to ensure that all created VDF channels are in the disable state.
- Before the MI_VDF_Uninit call, you need to ensure that the results of all created VDF channels are released.

➤ Example

See [MI_VDF_Init](#) For example.

➤ Related APIs

N/A.

2.2.3 MI_VDF_CreateChn

➤ Functions

Create a video detection (MD/OD/VG) channel.

➤ Syntax

```
MI_S32 MI_VDF_CreateChn(MI_VDF_CHANNEL VdfChn, const MI\_VDF\_ChnAttr\_t* pstAttr);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	Specify the video detection channel number.	Input
pstAttr	Set the video channel property value.	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- VdfChn cannot be specified repeatedly.
- VdfChn valid range: $0 \leq \text{VdfCh} < \text{MI_VDF_CHANNEL_MAX}$.
- MI_VDF_CHANNEL_MAX is defined in header file mi_vdf_datatype.h.

➤ Example

N/A.

➤ Related APIs

[MI_VDF_DestroyChn](#)

2.2.4 MI_VDF_DestroyChn

➤ Functions

Destroy the created video detection channel.

➤ Syntax

```
MI_S32 MI_VDF_DestroyChn(MI_VDF_CHANNEL VdfChn);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- VdfChn cannot be specified repeatedly.
- VdfChn valid range: $0 \leq \text{VdfCh} < \text{MI_VDF_CHANNEL_MAX}$.

- MI_VDF_CHANNEL_MAX is defined in header file mi_vdf_datatype.h.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.5 MI_VDF_SetChnAttr

➤ Functions

Set the video detection channel attributes.

➤ Syntax

```
MI_S32 MI_VDF_SetChnAttr(MI_VDF_CHANNEL VdfChn, const MI\_VDF\_ChnAttr\_t* pstAttr);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
pstAttr	Source port configuration information data structure pointer.	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- VdfChn must be a video detection channel that has been created.
- This interface specifies the value of the dynamic attribute of the video detection channel (stMdAttr.stMdDynamicParamsIn / stMdAttr.stOdDynamicParamsIn/ stMdAttr.stVgAttr in [MI_VDF_ChnAttr_t](#)).

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.6 MI_VDF_GetChnAttr

➤ Functions

Get the video detection channel attributes.

➤ Syntax

```
MI_S32 MI_VDF_GetChnAttr(MI_VDF_CHANNEL VdfChn, MI\_VDF\_ChnAttr\_t* pstAttr);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
pstAttr	Used to save the returned video detection channel	Output

	attribute value	
--	-----------------	--

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

N/A.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.7 MI_VDF_EnableSubWindow

➤ Functions

Enable/disable the specified video detection channel sub-window.

➤ Syntax

```
MI_S32 MI_VDF_EnableSubWindow(MI_VDF_CHANNEL VdfChn, MI_U8 u8Col, MI_U8 u8Row,
MI_U8 u8Enable);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
u8Col	The row address of the sub window (reserved, not used)	Input
u8Row	Sub-window column address (reserved, not used)	Input
u8Enable	Sub-window enable control flag	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: Mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- u8Col and u8Row are reserved for upward compatibility of API interface, and have no practical effect. You can set them to 0 directly.
- There are currently three working modes supported: MD, OD, and VG.
- The [MI_VDF_Run/MI_VDF_Stop](#) function acts on all video detection channels in the same working mode (MD/OD/VG), and the [MI_VDF_EnableSubWindow](#) function acts on a single video detection channel. To run a single VdfChn, call MI_VDF_Run function and MI_VDF_EnableSubWindow (u8Enable = True). Note that when you call MI_VDF_Stop function or MI_VDF_EnableSubWindow (u8Enable = False), the video detection channel will stop.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.8 MI_VDF_Run

➤ Functions

Start running video detection working mode.

➤ Syntax

```
MI_S32 MI_VDF_Run(MI\_VDF\_WorkMode\_e enWorkMode);
```

➤ Parameters

Parameter Name	Description	Input/Output
enWorkMode	Video detection (MD/OD/VG) working mode.	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: `mi_sys.h`, `mi_md.h`, `mi_od.h`, `mi_vg.h`, `mi_vdf.h`
- Library File: `libOD_LINUX.a`, `libMD_LINUX.a`, `libVG_LINUX.a`, `libmi_vdf.a`

➤ Note

- The work mode only supports MD/OD/VG.
- `MI_VDF_Run`/`MI_VDF_Stop` function acts on all video detection channels in the same working mode (MD/OD/VG).

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.9 MI_VDF_Stop

➤ Functions

Stop running video detection (MD/OD/VG) working mode.

➤ Syntax

```
MI_S32 MI_VDF_Stop(MI\_VDF\_WorkMode\_e enWorkMode);
```

➤ Parameters

Parameter Name	Description	Input/Output
enWorkMode	Video detection (MD/OD/VG) working mode.	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

- The work mode only supports MD/OD/VG.
- MI_VDF_Run/MI_VDF_Stop function acts on all video detection channels in the same working mode (MD/OD/VG).

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.10 MI_VDF_GetResult

➤ Functions

Get the detection result of the specified video detection channel.

➤ Syntax

```
MI_S32 MI_VDF_GetResult(MI_VDF_CHANNEL VdfChn, MI\_VDF\_Result\_t* pstVdfResult, MI_S32 s32MilliSec);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
pstVdfResult	Save the returned video detection results	Output
s32MilliSec	Input time parameter(reserved, not used currently)	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: `mi_sys.h`, `mi_md.h`, `mi_od.h`, `mi_vg.h`, `mi_vdf.h`
- Library File: `libOD_LINUX.a`, `libMD_LINUX.a`, `libVG_LINUX.a`, `libmi_vdf.a`

➤ Note

- VdfChn must be a video detection channel that has been created.
- s32MilliSec is reserved for upward compatibility of API interface, and has no practical effect. You can set it to 0 directly.
- The pointer used to save the returned result cannot be Null.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.11 MI_VDF_PutResult

➤ Functions

Release the detection result of the specified video detection channel.

➤ Syntax

```
MI_S32 MI_VDF_PutResult(MI_VDF_CHANNEL VdfChn, MI\_VDF\_Result\_t* pstVdfResult);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
pstVdfResult	Specify the result parameters that need to release the video channel	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

VdfChn must be a video detection channel that has been created.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.12 MI_VDF_GetLibVersion

➤ Functions

Obtain the MD/OD/VG library version number.

➤ Syntax

```
MI_S32 MI_VDF_GetLibVersion(MI_VDF_CHANNEL VdfChn, MI_U32* u32VDFVersion);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
u32VDFVersion	Pointer for saving the version number (cannot be Null).	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

N/A.

➤ Example

N/A.

➤ Related APIs

N/A.

2.2.13 MI_VDF_GetDebugInfo

➤ Functions

Get the MD/OD/VG debug info.

➤ Syntax

```
MI_S32 MI_VDF_GetDebugInfo(MI_VDF_CHANNEL VdfChn, MI_VDF_DebugInfo_t *pstDebugInfo);
```

➤ Parameters

Parameter Name	Description	Input/Output
VdfChn	The video detection channel number that has been created.	Input
u32VDFVersion	Pointer for saving the VDF debug info (cannot be Null)	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see error code for details

➤ Dependency

- Header File: mi_sys.h, mi_md.h, mi_od.h, mi_vg.h, mi_vdf.h
- Library File: libOD_LINUX.a, libMD_LINUX.a, libVG_LINUX.a, libmi_vdf.a

➤ Note

Only supports VG mode.

➤ Example

N/A.

➤ Related APIs

N/A.

3. DATA TYPE

3.1. VDF Structure List

The relevant data types, data structures, and unions are defined as follows:

Data structure	Definition
MI_VDF_WorkMode_e	An enumerated type that defines the working mode of VDF
MI_VDF_Color_e	Define the enumerated type of the video detection channel input source
MI_VDF_ODWindow_e	Enumerated type of the number of child windows of the screen when defining OD
MI_MD_Result_t	Structure that defines the MD result
MI_OD_Result_t	Structure that defines the OD result
MI_VG_Result_t	Structure that defines the VG result
MI_VDF_Result_t	Structure that defines the corresponding result of the VDF working mode
MI_VDF_MdAttr_t	Structure that defines MD channel attributes
MI_VDF_OdAttr_t	Structure that defines OD channel attributes
MI_VDF_VgAttr_t	Structure that defines VG channel attributes
MI_VDF_ChnAttr_t	Structure that defines the channel's working mode properties
MDRST_STATUS_t	Structure that defines a motion detection result of a detection sub-window region
MI_MD_ResultSize_t	Structure that defines the length of the MD result

3.2. VDF Structure Description

3.2.1 MI_VDF_WorkMode_e

➤ Description

Define VDF work mode enumeration type.

➤ Definition

```
typedef enum
{
    E_MI_VDF_WORK_MODE_MD = 0,
    E_MI_VDF_WORK_MODE_OD,
    E_MI_VDF_WORK_MODE_VG,
    E_MI_VDF_WORK_MODE_MAX
}MI_VDF_WorkMode_e;
```

➤ Members

Member Name	Description
E_MI_VDF_WORK_MODE_MD	MD working mode
E_MI_VDF_WORK_MODE_OD	OD working mode
E_MI_VDF_WORK_MODE_VG	VG working mode
E_MI_VDF_WORK_MODE_MAX	Error code identification of working mode

➤ Note

N/A.

➤ Related API and Types

N/A.

3.2.2 MI_VDF_Color_e

➤ Description

Define the enumerated type of the video detection channel input source

➤ Definition

```
typedef enum
{
    E_MI_VDF_COLOR_Y = 1,
    E_MI_VDF_COLOR_MAX
} MI_VDF_Color_e;
```

➤ Members

Member Name	Description
E_MI_VDF_COLOR_Y	Correct identification of the input source type of the video detection channel
E_MI_VDF_COLOR_MAX	Video detection channel input source type error code identification

➤ Note

N/A.

➤ Related API and Types

N/A.

3.2.3 MI_VDF_ODWindow_e

➤ Description

Enumerated type of the number of child windows of the screen when defining OD

➤ Definition

```
typedef enum
```

```

{
    E_MI_VDF_ODWINDOW_1X1 = 0,
    E_MI_VDF_ODWINDOW_2X2,
    E_MI_VDF_ODWINDOW_3X3,
    E_MI_VDF_ODWINDOW_MAX
} MI_VDF_ODWindow_e;

```

➤ Members

Member Name	Description
E_MI_VDF_ODWINDOW_1X1	The OD screen is divided into 1 sub-window
E_MI_VDF_ODWINDOW_2X2	The OD screen is divided into 2x2 sub-windows
E_MI_VDF_ODWINDOW_3X3	The OD screen is divided into 3x3 sub-windows
E_MI_VDF_ODWINDOW_MAX	Error code identification of the number of OD screen sub-windows

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.4 MI_MD_Result_t

➤ Description

A structure that defines the MD result.

➤ Definition

```
typedef struct MI_MD_Result_s
{
    MI_U64 u64Pts;          //The PTS of Image

    MI_U8 u8Enable;         // =1 indicates that the result value is valid

    MI_MD_ResultSize_t stSubResultSize;

    MDRST\_STATUS\_t* pstMdResultStatus; //The MD result of Status

    MDSAD\_DATA\_t* pstMdResultSad; //The MD result of SAD

    MDOBJ\_DATA\_t* pstMdResultObj; //The MD result of Obj

}MI_MD_Result_t;
```

➤ Members

Member Name	Description
u64Pts	Image display timestamp
u8Enable	Indicates whether the channel is enabled
u8Reading	Indicates that the result is being read by the application layer
stSubResultSize	Describe the Size of the Sad , Obj , and ReasultStauts substructures returned by the MD
pstMdResultStatus	Describe whether motion is detected in each area of the MD
pstMdResultSad	Describe the Sad value of the MD
pstMdResultObj	Describe the CCL value of the MD

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.5 MI_OD_Result_t

➤ Description

The structure that defines the OD result.

➤ Definition

```
typedef struct MI_OD_Result_s
{
    MI_U8 u8Enable;

    MI_U8 u8WideDiv;    //The number of divisions of window in horizontal direction
    MI_U8 u8HightDiv;   //The number of divisions of window in vertical direction
    MI_U8 u8DataLen;    //OD detect result readable size
    MI_U64 u64Pts;      //The PTS of Image

    MI_S8 u8RgnAlarm[3][3]; //The OD result of the sub-window
    MI_S8 s8OdStatus;    //The OD detect status
}MI_OD_Result_t;
```

➤ Members

Member name	description
u8Enable	Indicates whether the channel is enabled
u8WideDiv	Get the number of windows in the horizontal direction of the OD
u8HightD iv	Get the number of windows in the vertical direction of the OD
u8DataLen	Set the readable size of the OD test results
u64Pts	Image display time
u8RgnAlarm[3][3]	OD sub window information result 0: The window is not blocked

Member name	description
	1: The window is blocked 2: Insufficient window feature -1: Fail
s8OdStatus	1. s8OdStatus = 0: No od detected 2. s8OdStatus = 1: Od detected 3. s8OdStatus = -1: Run od error

➤ Note

If the user needs to obtain the value of s8OdStatus, it can be judged by the status value 0 or 1. When it is -1, it means that OD has failed to run, and MI_VDF_GetResult failed, so that the value of s8OdStatus cannot be obtained.

➤ Related API and Data Types

N/A.

3.2.6 MI_VG_Result_t

➤ Description

The structure that defines the VG result.

➤ Definition

```
typedef struct _MI_VgResult_t
{
    int32_t alarm[MAX_NUMBER];

    int32_t alarm_cnt;

    MI_VgBoundingBox_t bounding_box[20];
} MI_VgResult_t;
```

➤ Members

Member Name	Description
alarm	Describe the test results of VG #define MAX_NUMBER 4
alarm_cnt	Describe the number of VG alerts
bounding_box	Describe the object information that triggered the VG alarm

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.7 [MI_VDF_Result_t](#)

➤ Description

A structure that defines the result of the VDF working mode.

➤ Definition

```
typedef struct MI_VDF_Result_s
{
    MI\_VDF\_WorkMode\_e enWorkMode;

    VDF_RESULT_HANDLE handle;

    union
    {
```

```
MI_MD_Result_t stMdResult;  
  
MI_OD_Result_t stOdResult;  
  
MI_VG_Result_t stVgResult;  
  
};  
  
}MI_VDF_Result_t;
```

➤ Members

Member name	description
enWorkMode	VDF working mode (MD/OD/VG)
Handle	Save result handle
stMdResult	Structure describing the MD result
stOdResult	Structure describing the OD result
stVgResult	Structure describing the VG result

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.8 MI_VDF_MdAttr_t

➤ Description

Structure that defines MD channel properties

➤ Definition

```
typedef struct MI_VDF_MdAttr_s
```

```

{
    MI_U8 u8Enable;

    MI_U8 u8MdBufCnt;

    MI_U8 u8VDFIntvl;

    MI_U32 u32RstBufSize;

    MI_MD_ResultSize_t stSubResultSize;

    MDCCL\_ctrl\_t ccl_ctrl;

    MI\_MD\_static\_param\_t stMdStaticParamsIn;

    MI\_MD\_param\_t stMdDynamicParamsIn;

}MI_VDF_MdAttr_t;

```

➤ Members

Member name	description
u8Enable	Indicates whether the channel is enabled
u8MdBufCnt	Set the number of MD results that can be cached MD result cache number range: [1, 8] static attribute
u8VDFIntvl	Detection interval value range: [0, 29] , in frames, dynamic attribute
u32RstBufSize	The total size of the MD return results
stSubResultSize	MD returns the result of each substructure (Sad value, CCL value, ResultStatus) size
Ccl_ctrl	c cl _ctrl property setting
stMdStaticParamsIn	MD static attribute parameter setting
stMdDynamicParamsIn	MD dynamic attribute parameter setting

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.9 MI_VDF_OdAttr_t

➤ Description

Structure that defines OD channel attributes

➤ Definition

```
typedef struct MI_VDF_OdAttr_s
{
    MI_U8 u8Enable;

    MI_U8 u8OdBufCnt;

    MI_U8 u8VDFIntvl;

    MI_U32 u32RstBufSize;

    MI\_OD\_static\_param\_t stOdStaticParamsIn;

    MI\_OD\_param\_t stOdDynamicParamsIn;

}MI_VDF_OdAttr_t;
```

➤ Members

Member name	description
u8Enable	Indicates whether the channel is enabled
u8OdBufCnt	OD result cached number range: [1, 16] static attribute
u8VDFIntvl	Detection interval value range: [0, 29] , in frames, dynamic attribute
u32RstBufSize	The total size of the OD return results
stOdDynamicParamsIn	O D dynamic attribute parameter setting
stOdStaticParamsIn	O D static attribute parameter setting

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.10 MI_VDF_VgAttr_t

➤ Description

Structure that defines VG channel properties

➤ Definition

```
typedef struct MI_VDF_VgAttr_s
{
    MI_U8 u8Enable;

    MI_U8 u8VgBufCnt;

    MI_U8 u8VDFIntvl;

    MI_U32 u32RstBufSize;

    MI_U16 width;

    MI_U16 height;

    MI_U16 stride;

    float object_size_thd;

    uint8_t indoor;

    uint8_t function_state;

    uint16_t line_number;
```

```

MI_VgLine_t line[4];

MI_VgRegion_t vg_region;

MI_VgSet_t stVgParamsIn;

} MI_VDF_VgAttr_t;

```

➤ Members

Member name	description
u8Enable	Indicates whether the channel is enabled
u8VgBufCnt	VG result buffered number range: [1, 8] static attribute
u8VDFIntvl	Detection interval value range: [0, 29] , in frames, dynamic attribute
u32RstBufSize	The total size of the result returned by the VG
Width	Image width
Height	Image height
Stride	Image of stride
Object_size_thd	Decided to filter out the percentage of objects in the area of interest (If object_size_thd = 1 indicates that the object area is less than one percent of the region of interest in the image frame, it will be ignored.)
Indoor	Indoor or outdoor, 1- indoor, 0- outdoor
Function_state	Set a detection mode for virtual line segments and regional intrusions: 1) VG_VIRTUAL_GATE, indicates that the mode is a virtual line segment 2) VG_REGION_INVASION, indicates that the pattern is a zone intrusion
Line_number	Set the number of virtual segments, range: [1-4]
Line[4]	Show the structure of the virtual line, a maximum of four may be provided
Vg_region	Related parameters indicating regional invasion
stVgParamsIn	Vg attribute parameter structure, no need to set ,returned by API

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.11 MI_VDF_ChnAttr_t

➤ Description

A structure that defines the channel's working mode properties.

➤ Definition

```
typedef struct MI_VDF_ChnAttr_s
{
    MI\_VDF\_WorkMode\_e enWorkMode;

    union
    {
        MI\_VDF\_MdAttr\_t stMdAttr;
        MI\_VDF\_OdAttr\_t stOdAttr;
        MI\_VDF\_VgAttr\_t stVgAttr;
    };
}MI_VDF_ChnAttr_t;
```

➤ Members

Member name	description
enWorkMode	Working mode (motion detection , occlusion detection , electronic

	fence) static properties
stMdAttr	Motion detection attribute
stOdAttr	Occlusion detection attribute
stVgAttr	Electronic fence properties

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.12 MDRST_STATUS_t

➤ Description

A structure defining a motion detection result of a detection sub-window region

➤ Definition

```
typedef struct MDRST_STATUS_s
{
    MI_U8 *paddr;
} MDRST_STATUS_t;
```

➤ Members

Member Name	Description
paddr	Directed motion detection state buf, each area occupies 1byte 0- block does not detect motion, 255- block detects motion

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.2.13 MI_MD_ResultSet_t

➤ Description

Structure that defines the length of the MD result

➤ Definition

```
typedef struct MI_MD_ResultSet_s
{
    MI_U32 u32RstStatusLen;

    MI_U32 u32RstSadLen;

    MI_U32 u32RstObjLen;
}MI_MD_ResultSet_t;
```

➤ Members

Member Name	Description
u32RstStatusLen	Describe the length of the MD detected motion state value
u32RstSadLen	Describe the length of the Sad value of MD
u32RstObjLen	Describe the length of the CCL value of MD

➤ Note

N/A.

➤ Related API and Data Types

N/A.

3.3. MD Structure List

数据类型	定义
MDMB_MODE_e	Macro block size of enumeration value
MDSAD_OUT_CTRL_e	Enumeration value of the SAD output format
MDALG_MODE_e	The operation mode enumeration value of the CCL connected region can be used for CCL operation according to the foreground result or the SAD result.
MDCCL_ctrl_t	Parameter structure that controls CCL operation
MDPreproc_ctrl_t	Control Parameter for MI_MD_Preproc running
MDblock_info_t	Define the ROI coordinate structure of a block.
MDPoint_t	Coordinate structure
MDROI_t	MD detection area structure
MDSAD_DATA_t	The structure of the MI_MD_ComputeImageSAD function output.
MDOBJ_t	Define the information of the connected area: the area and the coordinate position of the smallest bounding rectangle
MDOBJ_DATA_t	Structure of CCL output
MI_MD_IMG_t	The image source structure of motion detection is divided into physical and virtual memory address pointers.
MI_MD_IMG64_t	The image source structure of motion detection is divided into 64 bit physical memory address and virtual memory address pointer.
MI_MD_static_param_t	MD static parameter setting structure
MI_MD_param_t	MD dynamic parameter setting structure

3.4. MD Structure Description

3.4.1 MDMB_MODE_e

➤ Description

The enumerated size of Micro-block.

➤ Definition

```
typedef enum MDMB_MODE_E
{
    MDMB_MODE_MB_4x4    = 0x0,
    MDMB_MODE_MB_8x8    = 0x1,
    MDMB_MODE_MB_16x16  = 0x2,
    MDMB_MODE_BUTT
} MDMB_MODE_e;
```

➤ Members

Member Name	Description
MDMB_MODE_MB_4x4	Use 4x4 Micro-block
MDMB_MODE_MB_8x8	Use 8x8 Micro-block
MDMB_MODE_MB_16x16	Use 16x16 Micro-block

3.4.2 MDSAD_OUT_CTRL_e

➤ Description

The enumeration value of the SAD output format.

➤ Definition

```
typedef enum MDSAD_OUT_CTRL_E
{
    MDSAD_OUT_CTRL_16BIT_SAD = 0x0,
    MDSAD_OUT_CTRL_8BIT_SAD  = 0x1,
    MDSAD_OUT_CTRL_BUTT
} MDSAD_OUT_CTRL_e;
```

➤ Members

Member Name	Description
MDSAD_OUT_CTRL_16BIT_SAD	16 bit Output
MDSAD_OUT_CTRL_8BIT_SAD	8 bit Output

3.4.3 MDALG_MODE_e

➤ Description

The operation mode enumeration value of the CCL connected region can be CCL calculated according to the foreground result or the SAD result

➤ Definition

```
typedef enum MDALG_MODE_E
{
    MDALG_MODE_FG    = 0x0,
    MDALG_MODE_SAD   = 0x1,
    MDALG_MODE_FRAMEDIFF = 0x2,
    MDALG_MODE_BUTT
} MDALG_MODE_e;
```

➤ Members

Member Name	Description
MDALG_MODE_FG	Foreground mode
MDALG_MODE_SAD	SAD Mode
MDALG_MODE_FRAMEDIFF	FrameDifference Mode

3.4.4 MDCCL_ctrl_t

➤ Description

Control the parameter structure of the CCL execution.

➤ Definition

```
typedef struct MDCCL_ctrl_s
{
    uint16_t u16InitAreaThr;
    uint16_t u16Step;
} MDCCL_ctrl_t;
```

➤ Members

Member Name	Description
u16InitAreaThr	Area threshold(This value is not necessarily useful,but it recommended value is 8)
u16Step	Increased value of each threshold(This value is not necessarily useful,but it recommended value is 2)

3.4.5 MDPreproc_ctrl_t

➤ Description

Contol Parameter for MI_MD_Preproc running

➤ Definition

```
typedef struct MDPreproc_ctrl_s
{
    uint16_t u16Md_rgn_size;

    uint16_t u16Align;
} MDPreproc_ctrl_t;
```

➤ Members

Member Name	Description
u16Md_rgn_size	The threshold value of the size range of the moving area. If it is less than this threshold, the area is not calculated as a dynamic area
u16Align	Align restriction corresponding to CNN model (HC/HD:32ss align, FD:64align)

3.4.6 MDblock_info_t

➤ Description

Define the ROI coordinate structure of a block.

➤ Definition

```
typedef struct MDblock_info_s
{
    uint16_t st_x;
```

```

uint16_t st_y;

uint16_t end_x;

uint16_t end_y;

} MDblock_info_t;

```

➤ Members

Member Name	Description
st_x	Upper left X coordinate of ROI
st_y	Upper left y coordinate of ROI
end_x	Lower right X coordinate of ROI
end_y	Lower right y coordinate of ROI

3.4.7 MDPoint_t

➤ Description

Coordinate structure.

➤ Definition

```

typedef struct MDPoint_s
{
    uint16_t x;

    uint16_t y;

} MDPoint_t;

```

➤ Members

Member Name	Description
x	X Coordinate

y	Y Coordinate

3.4.8 MDROI_t

➤ Description

MD detection area structure.

➤ Definition

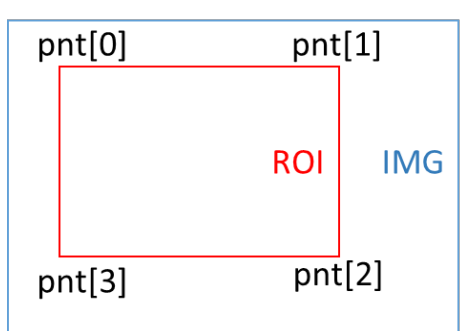
```
typedef struct MDROI_s
{
    uint8_t num;
    MDPoint\_t pnt[8];
} MDROI_t;
```

➤ Members

Member Name	Description
num	MD detection area points, currently only supported to set to 4 points
pnt[8]	Four-point coordinates (must be set to rectangle)

➤ Note

The requirement is set to a rectangle, num=4 , and the coordinates of the upper left corner are set clockwise in sequence, as shown in the figure.



3.4.9 MDSAD_DATA_t

➤ Description

The structure of the [MI_MD_ComputeImageSAD](#) function output.

➤ Definition

```
typedef struct MDSAD_DATA_s
{
    void *paddr;

    uint32_t stride;

    MDSAD\_OUT\_CTRL\_e enOutCtrl;
} MDSAD_DATA_t;
```

➤ Members

Member Name	Description
paddr	Memory address pointer for storing SAD results
stride	Image stride
enOutCtrl	Enumeration value of the SAD output format

3.4.10 MDOBJ_t

➤ Description

Define the information of the connected area: the area and the coordinate position of the smallest bounding rectangle.

➤ Definition

```
typedef struct MDOBJ_s
{
    uint32_t u32Area;

    uint16_t u16Left;

    uint16_t u16Right;

    uint16_t u16Top;

    uint16_t u16Bottom;
} MDOBJ_t;
```

➤ Members

Member name	description
u32Area	Total number of pixels in a single connected area
u16Left	The top left x coordinate of the smallest rectangle
u16Right	The lower right x coordinate of the smallest rectangle
u16Top	The upper left y coordinate of the smallest rectangle
u16Bottom	The upper left y coordinate of the smallest rectangle

3.4.11 MDOBJ_DATA_t

➤ Description

The structure of the [MI_MD_CCL](#) output.

➤ Definition

```
typedef struct MDOBJ_DATA_s
{
    uint8_t u8RegionNum;

    MDOBJ\_t *astRegion;

    uint8_t indexofmaxobj;

    uint32_t areaofmaxobj;

    uint32_t areaoftotalobj;

} MDOBJ_DATA_t;
```

➤ Members

Member name	description
u8RegionNum	Number of connected areas.
astRegion	Information about the connected area: the area and the coordinate position of the smallest bounding rectangle. The maximum allowable number is 255.
Indexofmaxobj	Maximum area connected area index value.
Areaofmaxobj	Maximum area of connected area.
Areaoftotalobj	The area sum-up of all connected areas.

3.4.12 [MI_MD_IMG_t](#)

➤ Description

The image source structure of motion detection is divided into physical and virtual memory address pointers.

➤ Definition

```
typedef struct MI_MD_IMG_s
{
    void *pu32PhyAddr;

    uint8_t *pu8VirAddr;
} MI_MD_IMG_t;
```

➤ Members

Member Name	Description
pu32PhyAddr	Physical memory address pointer, default is NULL
pu8VirAddr	Virtual memory address pointer

3.4.13 MI_MD_IMG64_t

➤ Description

The image source structure of motion detection is divided into 64 bit physical memory address and virtual memory address pointer.

➤ Definition

```
typedef struct MI_MD_IMG64_s
{
    uint64_t u64PhyAddr;

    uint8_t *pu8VirAddr;
} MI_MD_IMG64_t;
```

➤ Members

Member Name	Description
-------------	-------------

u64PhyAddr	Physical memory address
pu8VirAddr	Virtual memory address pointer

3.4.14 MI_MD_static_param_t

➤ Description

MD static parameter setting structure.

➤ Definition

```
typedef struct MI_MD_static_param_s
{
    uint16_t width;

    uint16_t height;

    uint8_t color;

    uint32_t stride;

    MDMB\_MODE\_e mb_size;

    MDSAD\_OUT\_CTRL\_e sad_out_ctrl;

    MDROI\_t roi_md;

    MDALG\_MODE\_e md_alg_mode;
} MI_MD_static_param_t;
```

➤ Members

Member name	description
Width	Input image width
Height	Input image is high
Stride	Input image of stride
Color	MD input image type

Member name	description
Mb_size	Macro block size enumeration value
Sad_out_ctrl	Enumeration value of the SAD output format
Roi_md	MD detection area structure
Md_alg_mode	Operation mode enumeration value of CCL connected area

3.4.15 MI_MD_param_t

➤ Description

MD dynamic parameter setting structure.

➤ Definition

```
typedef struct MI_MD_param_s
{
    uint8_t sensitivity;

    uint16_t learn_rate;

    uint32_t md_thr;

    uint32_t obj_num_max;

    uint8_t LSD_open;
} MI_MD_param_t;
```

➤ Members

Member name	description
Sensitivity	Algorithm sensitivity, range [10, 20, 30, 100] , the larger the value, the more sensitive, the sensitivity of the input is not A multiple of 10 , when the feedback is calculated , there may be a value that is not originally

Member name	description
	entered , there will be +-1 deviation. Only for VG mode.
Learn_rate	There are different setting standards for different modes: ✓ MDALG_MODE_FG : Unit is millisecond, range [1000-30000], used to control how long the front-end object stops moving before it is used as the background picture. ✓ MDALG_MODE_SAD : Range [1, 255], background update weight ratio, recommended setting value: 128.
Md_thr	When calling MI_MD_CCL connected component mark operation, as the threshold value to determine the effective movement, there are different setting standards for different modes: ✓ MDALG_MODE_FG : Set to percentage (%), range [0, 99], judge whether MB alarm threshold value, ex. if MB size 8 * 8, MD_Thr = 50, if more than 32 pixels in MB are foreground, the MB is judged as valid. ✓ MDALG_MODE_SAD : If MB size 8 * 8, MD_Thr = 50, when the average pixel difference in MB exceeds 50, the MB is judged as valid.
Obj_num_max	CCL connected area number limit value
LSD_open	Decide whether to use MI or not_ MD_LightSwitchDetect. The range is [0, 1]. If the LightSwitchDetect() function is called when LSD_open=0, it has no effect.

3.5. OD Structure List

Enumeration	
MI_OD_WIN_STATE	OD detection window results
ODColor_e	Type of OD data source input
ODWindow_e	Type of OD detection window
Structure	
ODPoint_t	Coordinate structure

ODROI_t	OD detection area structure
MI_OD_IMG_t	The source structure of the occlusion detection image is divided into physical and virtual memory address pointers.
MI_OD_IMG64_t	The source structure of the occlusion detection image is divided into 64 bit physical memory address and virtual memory address pointer.
MI_OD_static_param_t	OD static parameter setting structure
MI_OD_param_t	OD dynamic parameter setting structure

3.6. OD Structure Description

3.6.1 MI_OD_WIN_STATE

➤ Description

The result of the OD detection window.

➤ Definition

```
typedef enum _MI_OD_WIN_STATE
{
    MI_OD_WIN_STATE_NON_TAMPER = 0,
    MI_OD_WIN_STATE_TAMPER = 1,
    MI_OD_WIN_STATE_NO_FEATURE = 2,
    MI_OD_WIN_STATE_FAIL = -1,
} MI_OD_WIN_STATE;
```

➤ Members

Member name	description
MI_OD_WIN_STATE_NON_TAMPER	Window is not obscured
MI_OD_WIN_STATE_TAMPER	Window is occluded

MI_OD_WIN_STATE_NO_FEATURE	Insufficient window features
MI_OD_WIN_STATE_FAIL	failure

3.6.2 ODColor_e

➤ Description

The type of OD data source input.

➤ Definition

```
typedef enum
{
    OD_Y = 1,
    OD_COLOR_MAX
} ODColor_e;
```

➤ Members

Member Name	Description
OD_Y	y component in the YUV data source
OD_COLOR_MAX	Enter the maximum value of the image type

3.6.3 ODWindow_e

➤ Description

The type of OD detection window, the recommended value is OD_WINDOW_3X3 for testing.

➤ Definition

```
typedef enum
```

```
{  
    OD_WINDOW_1X1 = 0,  
    OD_WINDOW_2X2,  
    OD_WINDOW_3X3,  
    OD_WINDOW_MAX  
} ODWindow_e;
```

➤ Members

Member Name	Description
OD_WINDOW_1X1	1 window
OD_WINDOW_2X2	4 windows
OD_WINDOW_3X3	9 windows
OD_WINDOW_MAX	Maximum window type

3.6.4 ODPoint_t

➤ Description

Coordinate structure.

➤ Definition

```
typedef struct ODPoint_s  
{  
    uint16_t x;  
    uint16_t y;  
} ODPoint_t;
```

➤ Members

Member Name	Description
x	X coordinate
y	Y coordinate

3.6.5 ODROI_t

➤ Description

OD detection area structure.

➤ Definition

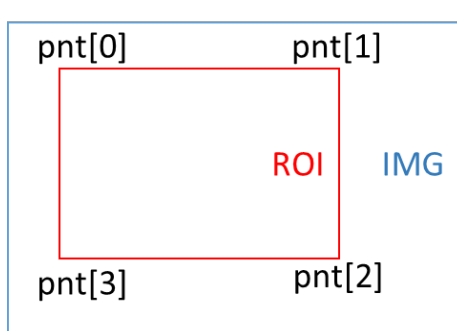
```
typedef struct ODROI_s
{
    uint8_t num;
    ODPoint\_t pnt[8];
} ODROI_t;
```

➤ Members

Member Name	Description
num	OD detection area points, currently only supported to set to 4 points
pnt[8]	Four-point coordinates (must be set to rectangle)

➤ Note

The requirement is set to a rectangle, num=4 , and the coordinates of the upper left corner are set clockwise in sequence, as shown in the figure.



3.6.6 MI_OD_IMG_t

➤ Description

The source structure of the occlusion detection image is divided into physical and virtual memory address pointers.

➤ Definition

```
typedef struct MI_OD_IMG_s
{
    void *pu32PhyAddr;
    uint8_t *pu8VirAddr;
} MI_OD_IMG_t;
```

➤ Members

Member Name	Description
pu32PhyAddr	Entity memory address pointer
pu8VirAddr	Virtual memory address pointer

3.6.7 MI_OD_IMG64_t

➤ Description

The source structure of the occlusion detection image is divided into 64 bit physical memory address and virtual memory address pointer.

➤ Definition

```
typedef struct MI_OD_IMG64_s
{
    uint64_t u64PhyAddr;
    uint8_t *pu8VirAddr;
} MI_OD_IMG64_t;
```

➤ Members

Member Name	Description
u64PhyAddr	Entity memory address
pu8VirAddr	Virtual memory address pointer

3.6.8 MI_OD_static_param_t

➤ Description

OD static parameter setting structure.

➤ Definition

```
typedef struct MI_OD_static_param_s
{
    uint16_t inImgW;
    uint16_t inImgH;
    uint32_t inImgStride;
    ODColor\_e nClrType;
```

```

ODWindow_e div;

ODROI_t roi_od;

int32_t alpha;

int32_t M;

int32_t MotionSensitivity;

} MI_OD_static_param_t;

```

➤ Members

Member name	description
inImgW	Input image width
inImgH	Input image is high
inImgStride	Input image stride
nClrType	OD input image type
Div	Type of OD detection window
Roi_od	OD detection area structure
Alpha	Controlling the learning rate of the reference image
M	How many images are updated once for reference images
MotionSensitivity	Mobile sensitivity setting

➤ Note

- Setting range Alpha : 0~10 , it is recommended to set 2 , it is not recommended to change.
- Setting range MotionSensitivity: 0~5 , set 5 to be sensitive to slight sway, easy to report; set 0 means better tolerance for slight sway, will not report, this side refers to the slight sway is wind swaying The recommended initial setting is 5 .
- M is recommended to set 120 , it is not recommended to change

3.6.9 MI_OD_param_t

➤ Description

OD dynamic parameter setting structure.

➤ Definition

```
typedef struct MI_OD_param_s
{
    int32_t thd_tamper;

    int32_t tamper_blk_thd;

    int32_t min_duration;
} MI_OD_param_t;
```

➤ Members

Member name	description
Thd_tamper	Image difference proportional threshold
Tamper_blk_thd	Image occlusion area number threshold value
Min_duration	Image difference duration threshold,The unit is frame cnt

➤ Note

- Set the range thd_tamper: 0~10. If thd_tamper=3, it means that more than 70% of the picture is occluded.
- Set the range tamper_blk_thd: MI_OD_Init type parameter corresponds to the window. If it is OD_WINDOW_3X3, the maximum number of tamper_blk_thd cannot exceed 9, that is, the range is 1~9.
- MI_OD_Init parameters such as window type is OD_WINDOW_3X3 (9 subareas) tamper_blk_thd value 4, when the number of sub-areas is blocked up to 4 MI_OD_Runtriggered only returns a value of 1.
- The larger the min_duration value, the longer it takes to detect occlusion.
- The sensitivity of MI_OD_Run can be adjusted by

setting tamper_blk_thd and min_duration. The recommended values for high, medium and low are as follows:

parameter name	high	in	low
Tamper_blk_thd	2	4	8
Min_duration	5	15	30

3.7. VG Structure List

Enumeration	
VgFunction	Detect mode enumeration value
VgRegion_Dir	Enumeration value of the direction of the zone intrusion
VgSize_Sensitively	Sensitivity parameter structure
VgDirection_State	Enumerated values of detection rules for regional intrusion
Structure	
MI_VG_Point_t	Coordinate point corresponding structure
MI_VgLine_t	Structure describing virtual segments and directions
MI_VgRegion_t	Describe the structure of setting up zone intrusion
MI_VgSet_t	Vg corresponds to the structure of the parameter settings
MI_VgResult_t	Corresponding structure of Vg detection result
MI_VgBoundingBox_t	Object size structure corresponding to Vg test results
MI_VgDetectThd	Vg establish background information structure parameter setting

3.8. VG Structure Description

3.8.1 VgFunction

➤ Description

The enumeration value of the detection mode.

➤ Definition

```
typedef enum _VgFunction
{
    VG_VIRTUAL_GATE    = 2,
    VG_REGION_INVASION = 3
} VgFunction;
```

➤ Members

Member name	description
VG_VIRTUAL_GATE	Indicates that the mode is a virtual line segment
VG_REGION_INVASION	Indicates that the pattern is a regional invasion

3.8.2 VgRegion_Dir

➤ Description

The enumeration value of the direction of the zone intrusion.

➤ Definition

```
typedef enum _VgRegion_Dir
{
    VG_REGION_ENTER    = 0,
    VG_REGION_LEAVING  = 1,
    VG_REGION_CROSS    = 2
} VgRegion_Dir;
```

➤ Members

Member name	description
VG_REGION_ENTER	Indicates that an alarm is triggered before entering the alarm zone
VG_REGION_LEAVING	Indicates that you want to leave the alert area to trigger an alert
VG_REGION_CROSS	Indicates that an alarm is triggered as soon as it crosses the alarm zone

3.8.3 VgSize_Sensitively

➤ Description

Sensitivity parameter structure.

➤ Definition

```
typedef enum _VgSize_Sensitively
{
    VG_SENSITIVELY_MIN    = 0,
    VG_SENSITIVELY_LOW    = 1,
    VG_SENSITIVELY_MIDDLE = 2,
    VG_SENSITIVELY_HIGH   = 3,
    VG_SENSITIVELY_MAX    = 4
} VgSize_Sensitively;
```

➤ Member

Member name	Description
VG_SENSITIVELY_MIN	Detect objects larger than 30%
VG_SENSITIVELY_LOW	Detect objects larger than 10%
VG_SENSITIVELY_MIDDLE	Detect objects larger than 5%

VG_SENSITIVELY_HIGH	Detect objects larger than 1%
VG_SENSITIVELY_MAX	Detect objects larger than 0.5%

3.8.4 VgDirection_State

➤ Description

Enumerated values of detection rules for regional intrusion.

➤ Definition

```
typedef enum _VgDirection_State
{
    VG_SPEC_DIRECTION_CLOSE = 0,
    VG_SPEC_DIRECTION_OPEN = 1
} VgDirection_State;
```

➤ Member

Member name	Description
VG_SPEC_DIRECTION_CLOSE	The closed result will distinguish the function of entering and leaving (Under the premise of setting VG_REGION_CROSS, only the alarm is detected, and there is no function of judging the direction.)
VG_SPEC_DIRECTION_OPEN	The opening result will distinguish the function of entering and leaving (under the premise of setting VG_REGION_CROSS, it can be judged whether the alarm direction is entering or leaving.)

3.8.5 MI_VG_Point_t

➤ Description

Describe the structure of the virtual line segment and direction.

➤ Definition

```
typedef struct _VG_Point_t
{
    int32_t x;
    int32_t y;
} MI_VG_Point_t;
```

➤ Members

Member Name	Description
x	X Coordinate
y	Y Coordinate

3.8.6 MI_VgLine_t

➤ Description

Describe the structure of the virtual line segment and direction.

➤ Definition

```
typedef struct _VG_Line_t
{
    MI_VG_Point_t px; //point x
    MI_VG_Point_t py; //point y
```

```

MI_VG_Point_t pdx; //point direction x

MI_VG_Point_t pdy; //point direction y

} MI_VgLine_t;

```

➤ Members

Member name	description
Px	First line point
Py	Second line point
Pdx	First direction point
Pdy	Second direction point

3.8.7 MI_VgRegion_t

➤ Description

Describe the structure of setting up zone intrusion.

➤ Definition

```

typedef struct _VG_Region_t
{
    MI_VG_Point_t p_one; //point one

    MI_VG_Point_t p_two; //point two

    MI_VG_Point_t p_three; //point three

    MI_VG_Point_t p_four; //point four


    int32_t region_dir; //Region direction;

    int32_t spec_dir_state;

} MI_VgRegion_t;

```

➤ Members

Member name	description
P_one	Describe the first point of the area
P_two	Describe the second point of the area
P_three	Describe the third point of the area
P_four	The fourth point of the description area
Region_dir	Set the direction of the area intrusion
spec_dir_state	Set whether to distinguish between entering and leaving when region_dir = VG_REGION_CROSS

3.8.8 MI_VgSet_t

➤ Description

Vg corresponds to the structure of the parameter settings.

➤ Definition

```
typedef struct _MI_VgSet_t
{
    //Common Information

    float object_size_thd;

    uint16_t line_number;

    uint8_t indoor;

    //Line info

    MI_VG_Point_t fp[4]; //First point
```

```
MI_VG_Point_t sp[4]; //Second point

MI_VG_Point_t fdp[4]; //First direction point

MI_VG_Point_t sdp[4]; //Second direction point


//Function

uint8_t function_state;


//Region info

MI_VG_Point_t first_p; //First point

MI_VG_Point_t second_p; //Second point

MI_VG_Point_t third_p; //Third point

MI_VG_Point_t fourth_p; //Fourth point


//Region direction

uint8_t region_direction;


//Magic_number

int32_t magic_number;


int32_t region_spdir_state;

} MI_VgSet_t;
```

➤ Members

Member name	description
Object_size_thd	Decided to filter out objects of interest to the area accounted for a percentage threshold
Line_number	Set the number of virtual segments, range: [1-4]

Member name	description
Indoor	Indoor or outdoor, 1- indoor, 0- outdoor
Fp[4]	The first point of the line array
Sp[4]	Second line point array
Fdp[4]	Third line segment point array
Sdp[4]	Fourth line segment point array
Function_state	Set virtual line segment and area intrusion detection mode
First_p	The first point in the invading area
Second_p	Second point in the invading area
Third_p	The third point of the invading area
Fourth_p	Invasion area is four points lower
Region_direction	Related parameters indicating regional invasion
Magic_number	Magic Number
region_spdir_state	The relevant rule direction parameters of the invading area

3.8.9 MI_VgResult_t

➤ Description

The structure corresponding to the Vg detection result.

➤ Definition

```
typedef struct _MI_VgResult_t
{
    int32_t alarm[4];

    int32_t alarm_cnt;

    MI_VgBoundingBox_t bounding_box[20];
}
```

```
} MI_VgResult_t;
```

➤ Members

Member Name	Description
alarm[4]	Vg alarm result
alarm_cnt	Vg alarm times
bounding_box	The object information for Vg trigger an alarm

3.8.10 MI_VgBoundingBox_t

➤ Description

Object size structure corresponding to Vg test results

➤ Definition

```
typedef struct _MI_VgBoundingBox_t
{
    int32_t up;
    int32_t down;
    int32_t left;
    int32_t right;
} MI_VgBoundingBox_t;
```

➤ Member

Member name	Description
up	The minimum y-coordinate information of the object
down	The maximum y-coordinate information of the

	object
left	The minimum x-coordinate information of the object
right	The maximum x-coordinate information of the object

3.8.11 MI_VgDetectThd

➤ Description

Vg establish background information structure parameter setting

➤ Definition

```
typedef struct _MI_VdDetectThd_t
{
    uint8_t function_switch;
    uint8_t detect_thd;
} MI_VgDetectThd;
```

➤ Member

Member name	Description
function_switch	Custom threshold to establish the functional status of the background (0-off, 1-on)
detect_thd	Custom threshold, range: [1-50]

4. ERROR CODE

Table 4-1 MI_VDF API Error Codes

Error Code	Macro Definition	Description
0xA0012001	MI_ERR_VDF_INVALID_DEVID	Device ID out of legal range

Error Code	Macro Definition	Description
0xA0012002	MI_ERR_VDF_INVALID_CHNID	Channel group number error or invalid area handle
0xA0012003	MI_ERR_VDF_ILLEGAL_PARAM	Argument out of legal range
0xA0012004	MI_ERR_VDF_EXIST	Repeat a device, channel, or resource that already exists in Create
0xA0012005	MI_ERR_VDF_UNEXIST	Trying to use a setting that does not exist in Destroyed
0xA0012006	MI_ERR_VDF_NULL_PTR	There are empty pointers in the function arguments
0xA0012007	MI_ERR_VDF_NOT_CONFIG	The module is not configured
0xA0012008	MI_ERR_VDF_NOT_SUPPORT	Unsupported parameters or function
0xA0012009	MI_ERR_VDF_NOT_PERM	Operation is not allowed, e.g. an attempt to modify static configuration parameters
0xA001200C	MI_ERR_VDF_NOMEM	Failed to allocate memory, e.g. insufficient system memory
0xA001200D	MI_ERR_VDF_NOBUF	Allocation cache failed, e.g. the requested data buffer is too large
0xA001200E	MI_ERR_VDF_BUF_EMPTY	No data in the buffer
0xA001200F	MI_ERR_VDF_BUF_FULL	Full data in buffer
0xA0012010	MI_ERR_VDF_NOTREADY	The system did not initialize or load the module
0xA0012011	MI_ERR_VDF_BADADDR	The address is illegal
0xA0012012	MI_ERR_VDF_BUSY	The system is busy
0xA0012013	MI_ERR_VDF_CHN_NOT_STARTED	The channel did not start.
0xA0012014	MI_ERR_VDF_CHN_NOT_STOPED	The channel has not stopped.
0xA0012015	MI_ERR_VDF_NOT_INIT	The module is not initialized
0xA0012016	MI_ERR_VDF_INITED	Module has been initialized
0xA0012017	MI_ERR_VDF_NOT_ENABLE	Channel or port is not enabled
0xA0012018	MI_ERR_VDF_NOT_DISABLE	Channel or port is not disabled

Error Code	Macro Definition	Description
0xA0012019	MI_ERR_VDF_TIMEOUT	Timeout
0xA001201A	MI_ERR_VDF_DEV_NOT_STARTED	The device did not start
0xA001201B	MI_ERR_VDF_DEV_NOT_STOPPED	The device is not stopped
0xA001201C	MI_ERR_VDF_CHN_NO_CONTENT	There is no information in the channel
0xA001201D	MI_ERR_VDF_NOVASAPCE	Failed to map virtual address
0xA001201E	MI_ERR_VDF_NOITEM	There is no record
0xA001201F	MI_ERR_VDF_FAILED	Unknown error

5. PROCFS INTRODUCTION

5.1. cat

➤ Debug info

```
# cat /proc/mi_modules/mi_shadow/mi_vdf0
```

➤ Debug info analysis

Record the current status of vdf and related attributes, which can be obtained dynamically for debugging and testing.

➤ Parameter Description

```

/mnt # cat /proc/mi_modules/mi_shadow/mi_vdf0
-----Common info for mi_vdf-----
ChnNum      EnChnNum    PassNum      InPortNum    OutPortNum    CollectSize
  8          3          1          1          1          0

-----CMDQ kickoff counter-----
DevId      current_buf_size    Peak_buf_size
  0          0          0

each dev buf info:
offset      length      used_flag      task_name

-----Common info formi_vdf only dump enabled chn-----
ChnId      PassNum      EnInPNum      EnOutPNum      MMAHeapName
  0          0          1          1      (null)
  1          0          1          1      (null)
  2          0          1          1      (null)

chnId      current_buf_size    Peak_buf_size    user_pid
  0          0          2000          815

each chn buf info:
offset      length      used_flag      task_name
ChnId      current_buf_size    Peak_buf_size    user_pid
  1          0          1000          815

each chn buf info:
offset      length      used_flag      task_name
ChnId      current_buf_size    Peak_buf_size    user_pid
  2          0          1000          815

each chn buf info:
offset      length      used_flag      task_name

-----chn0 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask    103323    131311    113375    121630
EnqueueInputTask          103738    131592    113675    122407
BarrierInputTask          103747    131828    113723    122416
CheckInputTaskStatus      103759    146785    122698    137559
DequeueInputTask          117069    146799    128701    137567
OnPollingAsyncOutputTaskConfig    -1          0          0          0
EnqueueAsyncOutputTask    30          90          70          72
BarrierAsyncOutputTask    33          99          77          79
CheckOutputTaskStatus      66      22731    5922    15176
DequeueOutputTask          11844    22743    14484    15191
FindMADispatch            11864    22768    14504    15217

-----chn1 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask    103506    106854    105941    106456
EnqueueInputTask          103512    107185    106138    106651
BarrierInputTask          103515    107193    106173    106717
CheckInputTaskStatus      103526    141451    117275    127816
DequeueInputTask          122113    141530    126349    127829
OnPollingAsyncOutputTaskConfig    -1          0          0          0
EnqueueAsyncOutputTask    29      235          66          50
BarrierAsyncOutputTask    33      2349    151          55
CheckOutputTaskStatus      69      24996    10665    21009
DequeueOutputTask          16218    25007    19742    21020
FindMADispatch            16250    25043    19790    21052

-----chn2 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask    103459    105979    105567    105907
EnqueueInputTask          103475    106608    105931    106407
BarrierInputTask          103480    106805    105969    106416
CheckInputTaskStatus      103489    127798    111808    121576
DequeueInputTask          118757    131181    122129    121584
OnPollingAsyncOutputTaskConfig    -1          0          0          0
EnqueueAsyncOutputTask    30      124          54          78
BarrierAsyncOutputTask    34      255          70          87
CheckOutputTaskStatus      82      24899    5904    14763
DequeueOutputTask          12559    24904    16805    14773
FindMADispatch            12607    24939    16990    14813

```

Table 1-1:

Parameter		Description
Common info for mi_vdf	ChnNum	The maximum channel number of the device
	EnChnNum	The enabled channel number of the device
	PassNum	The pass number of the device
	InPortNum	The inport number of the device
	OutPortNum	The outport number of the device
	CollectSize	The number of allocators contained in the AllocatorCollection

Parameter		Description
		corresponding to the device
Common info for mi_vdf only dump enabled chn	ChnId	Channel id
	PassNum	Pass number
	EnInPNum	The enabled input port number of the channel
	EnOutPNum	The enabled output port number of the channel
	MMAHeapName	If there is SetChnMMAConf, the value is the corresponding mma heap name; if it is not set, the value is NULL
	current_buf_size	The size of the currently requested memory
	Peak_buf_size	Peak memory usage
	user_pid	Current process pid

-----Input port common info for mi_vdf only dump enabled port-----										
ChnId	PassId	PortId	user_buf_quota	UsrInjectQ_cnt	BindInQ_cnt	TotalPendingBuf_size	usrLockedInjectCnt			
0	0	0	4	0	0	0	0			
1	0	0	4	0	0	0	0			
2	0	0	4	0	0	0	0			
ChnId	PassId	PortId	newPulseQ_cnt	nextToDoPulseQ_cnt	curWorkingQ_cnt	workingTask_cnt	lazzzyRewindTask_cnt			
0	0	0	0	0	0	0	0			
1	0	0	0	0	0	0	0			
2	0	0	0	0	0	0	0			
ChnId	PassId	PortId	Enable	bind_module_id	bind_module_name	bind_ChnId	bind_PortId	bind_Type	bind_Param	enable
0	0	0	1	12	mi_divp	2	0	1	0	1
1	0	0	1	12	mi_divp	2	0	1	0	1
2	0	0	1	12	mi_divp	2	0	1	0	1
ChnId	PassId	PortId	SrcFrmrate	DstFrmrate	GetFrame/Ms	FPS	FinishCnt	RewindCnt		
0	0	0	30/ 1	5/ 1	6/1201	4.99	32	0		
1	0	0	30/ 1	5/ 1	6/1200	5.00	27	0		
2	0	0	30/ 1	5/ 1	6/1201	4.99	22	0		

Table 1-2:

Parameter		Description
Input port	ChnId	The channel id of the input port
bind info	PassId	The pass id of the device
(only dump	PortId	The InputPort id of the device
enabled Input	user_buf_quota	The quota buff number of the InputPort
port)	UsrInjectQ_cnt	The buff number in UsrInjectBufQueue of the InputPort
	BindInQ_cnt	The buff number in BindInputBufQueue of the InputPort

Parameter		Description
	TotalPendingBuf_size	The buff total size in cur_working_input_queue of the InputPort
	usrLockedInjectCnt	The buff count currently taken by the user layer
	newPulseQ_cnt	The buff number in new_pulse_fifo_inputqueue of the InputPort
	nextTodoPulseQ_cnt	The buff number in next_todo_pulse_inputqueue of the InputPort
	curWorkingQ_cnt	The buff number in cur_working_input_queue of the InputPort
	workingTask_cnt	The buff number in input_working_tasklist of the InputPort
	lazzyRewindTask_cnt	The buff number in lazzy_rewind_inputtask_list of the InputPort (The task count that need to be retried)
	Enable	1: Enable 0: Disable
	bind_module_id	The module id of the output port binded with this input port
	bind_module_name	The module name of the output port binded with this input port (For example, the mod name here is divp, divp->vdf)
	bind_ChnId	The channel id of the output port binded with this input port
	bind_PortId	The id of the output port binded with this input port
	bind_Type	0x00000001: Frame (Frame mode, default work mode) 0x00000002: sw_low_latency (Low-latency work mode) 0x00000004: realtime mode (Hardware direct connection work mode) 0x00000008: hw autosync (Before and after handshake, the buffer size is consistent with the image resolution) 0x00000010: hw ring (Before and after handshake, ring buffer depth can be adjusted)
	bind_Param	Binded parameter

Parameter		Description
	enable	Enable
	SrcFrmrate	Source frame rate
	DstFrmrate	Target frame rate
	GetFrame/Ms	Actual frame rate/Time used
	FPS	Actual frame rate
	FinishCnt	Finished count
	RewindCnt	The task count that need to be retried

```

-----Output port common info for mi_vdf only for enabled port-----
ChnId  PassId  PortId  usrDepth  BufCntQuota  usrLockedCnt  totalOutPortInUsed  DrvBkRefFifoQ_cnt  DrvBkRefFifoQ_size
0      0      0      4         6             0             0                   0                   0
0      0      0      4         6             0             0                   0                   0
0      0      0      4         6             0             0                   0                   0
ChnId  PassId  PortId  UsrcGetFifoQ_cnt  UsrcGetFifoQ_size  UsrcGetFifoQ_seqnum  UsrcGetFifoQ_discardnum
0      0      0      0               0                   32                    0
0      0      0      0               0                   27                    0
0      0      0      0               0                   22                    0
ChnId  PassId  PortId  workingTask_cnt  finishedTask_cnt
0      0      0      0               0
0      0      0      0               0
0      0      0      0               0
ChnId  PassId  PortId  GetFrame/Ms  FPS  FinishCnt  RewindCnt  GetTotalCnt  GetOkCnt
0      0      0      5/1003      4.98  32         0          32         32
0      0      0      6/1200      5.00  27         0          27         27
0      0      0      6/1202      4.99  22         0          22         22
-----BindPeerInputPortList-----
ChnId  PassId  PortId  Enable  bind_module_id  bind_module_name  bind_ChnId  bind_PortId  bind_Type  bind_Param  enable

```

Table 1-3:

Parameter		Description
output port	ChnId	The channel id of the output port
bind info(only	PassId	The pass id of the device
dump enabled	PortId	The output port of the device
output port)	usrDepth	The maximum number of output port buf that the user can get
	BufCntQuota	The quota buff count of the OutputPort
	usrLockedCnt	The buffer count actually taken by the user layer from UsrcGetFifoBufQueue
	totalOutPortInUsed	The buffer count actually applied for by Output
	DrvBkRefFifoQ_cnt	The buffer number in DrvBkRefFifoQueue of the OutputPort

Parameter		Description
		(For special driver which need to refer back pre-processed buffer)
	DrvBkRefFifoQ_size	The buffer size occupied in the DrvBkRefFifoQueue of the OutputPort
	UsrGetFifoQ_cnt	The buffer count dumped in the UsrGetFifoBufQueue of the OutputPort
	UsrGetFifoQ_size	The buffer size dumped in the UsrGetFifoBufQueue of the OutputPort
	UsrGetFifoQ_seqnum	The buffer count got by the OutputPort
	UsrGetFifoQ_discardnum	The cumulative number of discarded original buffers that the OutputPort discarded due to new buffer requirements.
	finishedTask_cnt	The task count finished by the OutputPort
	GetFrame/Ms	Actual frame rate/time used
	FPS	Actual frame rate
	FinishCnt	Finished count
	RewindCnt	The task count that need to be retried
	GetTotalCnt	The buffer count got by the OutputPort
	GetOkCnt	The frame count of E_MI_FRC_OBTAIN

```

----- ** Dump all private info for VDF ** -----
VDF Verison: [Jul 13 2021 19:54:08]

----- DEV info for VDF -----
bInited 1 MDEnable 1 ODEnable 1 VGENable 1 YuvTaskExit 0 MDTaskExit 0 ODTTaskExit 0 VGTaskExit 0 DbgLevel 2

----- CHN common info for VDF only dump created chn -----
ChnId 0 eStatus 2 phandle 0xb53061a0 FrameCnt 0 FrameInv 0 YImgSize 101376 ImageQCnt 0
ChnId 1 eStatus 2 phandle 0xb0a08008 FrameCnt 0 FrameInv 0 YImgSize 101376 ImageQCnt 0
ChnId 2 eStatus 2 phandle 0xb53079b0 FrameCnt 0 FrameInv 0 YImgSize 101376 ImageQCnt 0

----- Input Port info for VDF only dump created chn -----
ChnId 0 InGetCnt 32 InGetFailCnt 340 InDropCnt 0 InDoneCnt 32 InFps 5.00
ChnId 1 InGetCnt 27 InGetFailCnt 294 InDropCnt 0 InDoneCnt 27 InFps 5.00
ChnId 2 InGetCnt 22 InGetFailCnt 242 InDropCnt 0 InDoneCnt 22 InFps 4.67

----- Output Port info for VDF only dump created chn -----
ChnId 0 OutGetCnt 32 OutGetFailCnt 32 OutDropCnt 0 OutDoneCnt 32 RstGetCnt 32 RstGetFailCnt 552 RstDropCnt 0 RstPutCnt 32 OutFps 5.00
ChnId 1 OutGetCnt 27 OutGetFailCnt 28 OutDropCnt 0 OutDoneCnt 27 RstGetCnt 27 RstGetFailCnt 459 RstDropCnt 0 RstPutCnt 27 OutFps 5.00
ChnId 2 OutGetCnt 22 OutGetFailCnt 22 OutDropCnt 0 OutDoneCnt 22 RstGetCnt 22 RstGetFailCnt 373 RstDropCnt 0 RstPutCnt 22 OutFps 5.00

----- MD Attr info for VDF only dump created chn [Ver: 20200210] -----
[====chn:0====]
[c_param]: Enable MdbufCnt VDFIntvl RstBufSize SubRstObjLen SubRstSadLen SubRstStsLen cc1_InitArea cc1_Step
           1 4 0 6304 3060 1584 1584 8 2
[s_param]: width height stride color mb_size sad_out_ctrl roi_md_num md_alg_mode
           352 288 352 1 1 1 4 1
           roi_md_num (pnt[0].x pnt[0].y) (pnt[1].x pnt[1].y) (pnt[2].x pnt[2].y) (pnt[3].x pnt[3].y)
           4 0 0 351 0 351 287 0 287
[d_param]: sensitivity learn_rate md_thr obj_num_max
           80 2000 16 0

----- OD Attr info for VDF only dump created chn [Ver: 20200108] -----
[====chn:1====]
[c_param]: Enable OdbufCnt VDFIntvl RstBufSize
           1 4 0 32
[s_param]: inImgW inImgH inImgStride nClrType alpha div roi_od_num M MotionSensit
           352 288 352 1 2 2 4 120 0
           roi_od_num (pnt[0].x pnt[0].y) (pnt[1].x pnt[1].y) (pnt[2].x pnt[2].y) (pnt[3].x pnt[3].y)
           4 0 0 351 0 351 287 0 287
[d_param]: thd_tamper tamper_blk_thd min_duration
           3 1 15

----- VG Attr info for VDF only dump created chn [Ver: 0] -----
[====chn:2====]
[c_param]: Enable VgBufCnt VDFIntvl RstBufSize
           1 4 0 340
[s_param]: width height stride obj_size_thd indoor func_state line_number
           352 288 352 3,00000 1 2
           line_number (p[0].x p[0].y) (p[1].x p[1].y) (p[2].x p[2].y) (p[3].x p[3].y)
           2 60, 150 160, 150 110, 190 110, 120 230, 220 330, 220 280, 190 280, 250

```

Table 1-4:

Parameter		Description
DEV info for VDF	VDF Verison	Version information
	bInited	The Vdf module is initialized or not. (0-Disable, 1-Enable)
	MDEnable	Enable MD or not. (0-Disable, 1-Enable)
	ODEnable	Enable OD or not. (0-Disable, 1-Enable)
	VGENable	Enable VG or not. (0-Disable, 1-Enable)
	YuvTaskExit	The thread that obtains yuv data exited or not. (0-Disable, 1-Enable)
	MDTaskExit	The MD algorithm thread exited or not. (0-Disable, 1-Enable)
	ODTaskExit	The OD algorithm thread exited or not. (0-Disable, 1-Enable)
	VGTaskExit	The VG algorithm thread exited or not. (0-Disable, 1-Enable)
	DbgLevel	Debug level of vdf module

Parameter		Description
		Parameter description: 1- disable 2- error 3- warn 4- info 5- debug 6- trace
CHN common info for VDF (only dump created chn)	ChnId	Channel id
	eStatus	0: Destroy 1: created 2: start 3: stop
	phandle	OD/MD/VG init handle pointer
	FrameCnt	Frame count (Not used currently)
	FrameInv	Interval frame (Not used currently)
	YImgSize	A yimage size(height*width) size sent from the previous stage
	ImageQCnt	The number of unprocessed yimages in the current queue
Input Port info for VDF (only dump created chn)	InGetCnt	The number of yimage obtained on the Input side
	InGetFailCnt	The number of yimage failures from the Input
	InDropCnt	The number of yimage discarded on the Input side
	InDoneCnt	The number of yimage data processed by the Input side
	InFps	The frame rate of the Image from the Input side
Output Port info for VDF (only)	OutGetCnt	The number of times the Output terminal gets the memory used to store the result of the operation

Parameter		Description	
dump created chn)	OutGetFailCnt	The number of times the Output terminal failed to get the memory used to store the result of the operation	
	OutDropCnt	The number of times the output side discards unprocessed memory	
	OutDoneCnt	The number of times that the output terminal writes the operation result to the memory successfully	
	RstGetCnt	The number of times the user succeeded in obtaining the algorithm result	
	RstGetFailCnt	The number of times the user failed to obtain the algorithm result	
	RstDropCnt	The number of times the algorithm result was discarded (discarded by new data)	
	RstPutCnt	The number of times the user released the memory used by the algorithm result	
	OutFps	The statistical frame rate of the output algorithm result	
MD Attr info for VDF (only dump created chn)	ChnId	Channel id	
	[c_param] Common parameter	Enable	Enable current channel or not
		MdBufCnt	The Buffer count that MD stores the result
		VDFIntvl	Not used currently
		RstBufSize	The size of the result returned at one time
		SubRstObjLen	The size of the CCL output result
		SubRstSadLen	The size of the SAD output result
		SubRstStsLen	The size of the checked result
		ccl_InitArea	Threshold value of CCL area area
		ccl_Step	The increase value of the threshold value for each increase in CCL
	[s_param]	width	The Image width input by MD
		height	The Image height input by MD

Parameter		Description	
	Static parameter		
		stride	The Image stride input by MD
		color	The Image color input by MD
		mb_size	MD macroblock size 1- MB_4x4 2- MB_8x8 3- MB_16x16
		sad_out_ctrl	MD SAD output format 1- 16BIT_SAD 2- 8BIT_SAD
		md_alg_mode	Algorithm mode of CCL Unicom area 1- FG mode 2- SAD mode
		roi_md_num	Number of boundary points of detection area
		Pnt[i].x	The x coordinate of each boundary point of the detection area
		Pnt[i].y	The y coordinate of each boundary point of the detection area
	Dynamic parameter	[d_param] sensitivity	Algorithm sensitivity, range [10,20,30,.....100], the larger the value, the more sensitive
		learn_rate	Unit milliseconds, range [1000,30000], used to control how long the front object stops moving before it is used as the background picture
		md_thr	The threshold value for judging the movement, there are different setting standards for different modes
		obj_num_max	CCL limit value for the number of connected

Parameter		Description	
			regions
OD Attr info for VDF (only dump created chn)	ChnId	Channel id	
	[c_param]	Enable	Enable current channel or not
	Common parameter	OdBufCnt	The Buffer count that OD stores the result
		VDFIntvl	Not used currently
		RstBufSize	The size of the result returned at one time
	[s_param]	inImgW	The Image width input by OD
	Static parameter	inImgH	The Image height input by OD
		inImgStride	The Image stride input by OD
		nClrType	OD input image type 3- y data
		alpha	Control the learning rate for generating reference images
		div	Type of OD detection window 1- 1 window 2- 4 windows 3- 9 windows
		M	How many images update the reference image once
		MotionSensit	Motion sensitivity setting
		roi_od_num	Number of boundary points of detection area
		pnt[i].x	The x coordinate of each boundary point of the detection area
		pnt[i].y	The y coordinate of each boundary point of the detection area
	[d_param]	thd_tamper	Image difference ratio threshold

Parameter		Description	
	Dynamic parameter	tamper_blk_thd	Threshold value of the number of occluded areas of the image
		min_duration	Image difference duration threshold
VG Attr info for VDF (only dump created chn)	ChnId	Channel id	
	[c_param]	Enable	Enable current channel or not
	Common parameter	VgBufCnt	The Buffer count that VG stores the result
		VDFIntvl	Not used currently
		RstBufSize	The size of the result returned at one time
	[s_param] Static parameter	width	The Image width input by VG
		height	The Image height input by VG
		stride	The Image stride input by VG
		obj_size_thd	The percentage size of the object in the area of interest
		indoor	Camera mounting area 1- outdoor 2- indoor
		func_state	Detection mode 3- VG_VIRTUAL_GATE, indicating that the mode is a virtual line segment. 4- VG_REGION_INVASION, indicating that the mode area is invaded.
		line_number	The number of virtual line segments, support 1 to 4 virtual line segments (The detection mode is effective for VG_VIRTUAL_GATE)
		p[i].x	X coordinate of line segment n (The detection mode is effective for

Parameter		Description	
			VG_VIRTUAL_GATE)
		p[i].y	Y coordinate of line segment n (The detection mode is effective for VG_VIRTUAL_GATE)
		pd[i].x	The first direction point of line segment n (The detection mode is effective for VG_VIRTUAL_GATE)
		pd[i].y	The second direction point of line segment n (The detection mode is effective for VG_VIRTUAL_GATE)
		region_dir	Set the direction of the region invasion. There are currently three directions to choose from: entering, leaving, and crossing. 1- REGION_ENTER: Entering the alarm region triggers an alarm. 2- REGION_LEAVING: Leaving the alarm region triggers an alarm. 3- REGION_CROSS: Crossing the alarm region triggers an alarm. (The detection mode is effective for VG_REGION_INVASION)
		Pnt[i].x	The x coordinate of the i-th point of the region. (The detection mode is effective for VG_REGION_INVASION)
		Pnt[i].y	The y coordinate of the i-th point of the region. (The detection mode is effective for VG_REGION_INVASION)

5.2. echo

echo help > /proc/mi_modules/mi_shadow/mi_vdf0

```

/ # echo help > /proc/mi_modules/mi_shadow/mi_vdf0
printk in _MI_SYS_IMPL_Common_WriteProc
----- _MI_SYS_IMPL_Common_Help start -----
entry_name=mi_vdf0 eModuleId=0x1 DevId=0x0 InputPortNum=0x1 OutputPortNum=0x1
use cat /proc/mi_modules/mi_vdf/mi_vdf0 to get attr supported by MI_SYS and this module
use echo help > /proc/mi_modules/mi_vdf/mi_vdf0 to get help
use echo dump_buffer parameter1 parameter2 parameter3 parameter4 parameter5 parameter6 > /proc/mi_modules/mi_vdf/mi_vdf0 to
dump buffer by MI_SYS
parameter1 chn_id: means channel id.
parameter2 port_type: value is "iport" or "oport".
parameter3 port_id: means port id of this channel.
parameter4 Queue_name: for input port ,only can be "UsrInject" or "BindInput"; for output port ,only support
"UsrGetFifo".
parameter5 path: result file is stored in which dir, should be absolute path.
parameter6 end_method: when will end dump process. only support "bufnum=xxx" , or "time=xxx" (here unit is ms)
or "start/end" pair.
use echo reset_counter [cmdq] [passid] > /proc/mi_modules/mi_vdf/mi_vdf0 to reset debug counters
----- _MI_SYS_IMPL_Common_Help end -----
----- ** _MI_VDF_OnHelp start ** -----
CMD: help
Sample: echo help > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: log_level
Param1: [0-6] 0-CLOSE 1-FATAL 2-ERROR 3-WARN 4-INFO 5-DEBUG 6-TRACE
Sample: echo log_level 5 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: sem_check
Param1: [0-1] 0-disable 1-enable
Sample: echo sem_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: en_workmode
Param1: [md/od/vg]
Param2: [0-1] 0-disable 1-enable
Sample: echo en_workmode md 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: en_chn
Param1: [chnId] chnId 0-63
Param2: [0-1] 0-disable 1-enable
Sample: echo en_chn 0 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: auto_check
Param1: [0-1] 0-disable 1-enable
Sample: echo auto_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: dump_all
Param1: [0-1] 0-disable 1-enable
Sample: echo dump_all 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: dump_image
Param1: [chnId] chnId 0-63
Param2: [dumpNum] 0-xxxx
Param3: [dumpPath] yourPath
Sample: echo dump_image 0 10 /mnt > /proc/mi_modules/mi_shadow/mi_vdf0
----- ** _MI_VDF_OnHelp end ** -----

```

Echo help view available commands.

Table 2-1:

Function	Modify the printing level
Command	echo log_level param1 > /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	The value of param1 is 0~6
Example	echo log_level 5 > mi_vdf0

Table 2-2:

Function	Set sem_check
----------	---------------

Command	echo sem_check param1> /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	The value of param1 is 0~1, 0-disable 1-enable
Example	echo sem_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0

Table 2-3:

Function	Enable/Disable en_workmode(od/md/vg)
Command	echo en_workmode param1 param2 > /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	param1: [md/od/vg]
	param2:[0-1] 0-disable 1-enable
Example	echo en_workmode md 0 > /proc/mi_modules/mi_shadow/mi_vdf0

Table 2-4:

Function	Enable/Disable vdf chn
Command	echo en_workmode param1 param2 > /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	param1: [chnId] chnId 0-63
	param2: [0-1] 0-disable 1-enable
Example	echo en_chn 0 0 > /proc/mi_modules/mi_shadow/mi_vdf0 //cat /proc/mi_modules/mi_shadow/mi_vdf0 again, you can see that ch0 of input/output has been closed

Table 2-5:

Function	Enable auto_check
Command	echo auto_check param1> /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	param1:[0-1] 0-disable 1-enable
Example	echo auto_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0

It is closed by default. After opening, cat /proc/mi_modules/mi_shadow/mi_vdf0 will print more to help you check such as YImageSize, chn and so on.

Table 2-6:

Function	Dump all information of vdf or not
Command	echo dump_all param1> /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	param1: [0-1] 0-disable 1-enable
Example	echo dump_all 1 > /proc/mi_modules/mi_shadow/mi_vdf0

Including function in Table 2-5.

Table 2-7:

Function	Dump yuv
Command	echo dump_image [param1, param2, param3] > /proc/mi_modules/mi_shadow/mi_vdf0
Parameter Description	param1: [chnId] chnId 0-15
	Param2: [dumpNum] 0-xxxx
	Param3: [dumpPath] yourPath
Example	echo dump_image 0 10 /mnt > /proc/mi_modules/mi_shadow/mi_vdf0

For example, dump chn0 10 YUV in /mnt directory:

```

# cd /mnt/
/mnt # ls
capture
mi.log
mi_sys.ko
mi_sys_2.ko
mi_test.ko
prog_cache
/mnt # ls -l
prog_sys
share
telnettest.sh
vdf[0]_image[1]_3452745.yuv
vdf[0]_image[2]_3452551.yuv
vdf[0]_image[3]_3452383.yuv
vdf[0]_image[4]_3452184.yuv
vdf[0]_image[5]_3451983.yuv
vdf[0]_image[6]_3451810.yuv
vdf[0]_image[7]_3451544.yuv
vdf[0]_image[8]_3451348.yuv
vdf[0]_image[9]_3451143.yuv
wifitestapp
yuv420sp_640x360.yuv
新建文本文档.txt

```